Appendix G: AI Usage Log

Student: Kamil Szlufik | 15377821 | 15377821@student.setu.ie

AI tool: Claude (Anthropic, 2025) — claude.ai

Period: January 2026 – April 2026

This appendix documents genuine uses of Claude AI throughout the development of PlayPal. Interactions covered four main areas: resolving specific technical blockers during development, styling and UI design decisions, structured learning sessions to understand the code being built, and assistance with the presentation and project documentation. All entries are genuine — either direct from conversation history or reconstructed accurately from session notes.

Blue boxes are my prompts. Green boxes are Claude's responses. Orange boxes describe what I actually did with the output and what was my own work.

---

1. Debugging

These entries came up during development when something stopped working and the error message alone was not enough to identify the root cause. I treated it like a more knowledgeable colleague — describe the exact symptom, paste the relevant code, get a diagnosis, then decide whether to apply it.

1.1 Firestore permission-denied immediately after account creation

My prompt

```
I'm getting a permission-denied error from Firestore right after
creating a new Firebase Auth account. Security rules look correct —
they allow authenticated users to write to their own document. The
account is definitely created because I can see it in Firebase
Console. The write happens on the very next line after
createUserWithEmailAndPassword(). It is not a rules problem — what is
going on?
```

AI response

```
Token propagation delay. Firebase Auth resolves the Future once the
account is
created, but the Firestore rules engine receives the auth token
asynchronously.
By the time the next line executes, Firestore hasn't received the token
yet and
treats the write as unauthenticated.

Fix — add a 2 second delay between account creation and first Firestore
write:

final cred = await
FirebaseAuth.instance.createUserWithEmailAndPassword(...);
await Future.delayed(const Duration(milliseconds: 2000)); // token
propagation
await _db.collection('users').doc(uid).set({...}); // now safe

This is a known Firebase behaviour. The delay gives the rules engine time
to
receive and process the auth token before evaluating any security rules.
```

What I did with this

Added the delay in sign_up_screen.dart. Also added a rollback — if the username is already taken after account creation, the auth account is deleted so there are no orphaned accounts in Firebase. Tested with ten consecutive signups. No more errors. Section 8.2 of the report.

1.2 Activity feed silently returning empty results

My prompt

```
My feed screen loads but shows nothing even though I can see the
documents in the Firebase console. No Flutter error, no exception
thrown — just an empty list. The query uses whereIn on uid and
orderBy on timestamp. The friend list is definitely populated. I have
been staring at this for two hours and cannot find anything wrong
with the query.
```

AI response

```
Two issues causing empty results:

1. Missing composite index. Firestore requires a composite index when
combining
   whereIn on one field with orderBy on another. Without it, Firestore
silently
   returns zero results — no error, no warning. Check Firebase Console
logs.
   There will be a direct clickable link to create the exact index
needed.

2. whereIn has a hard limit of 10 values per query. Any user with more
than 10
   friends will silently receive empty results. Fix: chunk the friend
list:

   for (int i = 0; i < friendUids.length; i += 10) {
     final chunk = friendUids.sublist(i, min(i+10, friendUids.length));
     final snap = await _db.collection('activity')
         .where('uid', whereIn: chunk)
         .orderBy('timestamp', descending: true)
         .limit(50).get();
     allActivities.addAll(snap.docs.map((d) => d.data()));
   }
   allActivities.sort((a,b) => ...) // sort merged results client-side
```

What I did with this

Created the composite index from the link in the Firebase Console logs. Implemented the chunking loop. Hit the same index issue again with the forum likeCount query and knew what to look for immediately. Section 8.6.

1.3 iPhone black screen after deployment

My prompt

```
Deployed to Firebase Hosting. Works perfectly on desktop Chrome.
Opened it on my iPhone — completely black screen, nothing renders at
all. No visible error. Flutter web, Safari on iOS. Files are
definitely loading in the network tab. Something is crashing silently
on mobile Safari.
```

AI response

```
Two likely causes for a black screen on Safari with Flutter web:

1. RenderFlex overflow. If any Row or Column has more children than fit
at
   375px width, Flutter's web renderer can crash silently and produce a
black
   screen. Reproduce in Chrome DevTools at 375px width — you'll see the
overflow
```

```
   error in the console there.

2. CanvasKit renderer compatibility. Flutter uses CanvasKit for web by
default
   which has known issues on mobile Safari. Switch to HTML renderer by
adding
   this to web/index.html before the Flutter loader script:
   <script>window.flutterConfiguration = { renderer: 'html' };</script>

Check your navigation bar specifically — if you have 6 buttons at 375px
that
is almost certainly the overflow causing the crash.
```

What I did with this

> *Found the overflow exactly in the nav bar — six buttons at 375px. Reduced to three visible buttons on mobile with the rest in a drawer. Added a 400px breakpoint where button padding and icon sizes scale down further for iPhone SE. Forced the HTML renderer. Tested on iPhone SE and iPhone 14. Section 8.8.*

1.4 Coming Soon and New Releases presets returning no games

My prompt

```
The Coming Soon, New Releases, and Best of 2026 presets on the home
screen are returning a completely empty grid. Top Rated and Most
Popular work fine. The date range is correct — I can verify it in the
URL being sent to RAWG. RAWG is just sending back zero results for
those queries.
```

AI response

```
The metacritic filter is the problem. You're sending metacritic: '1,100'
on all
queries which requires a Metacritic score. Upcoming and recently released
games
have not been reviewed yet so they have no score — they get filtered out
entirely.

Fix — make metacritic conditional on whether dates are active:

final queryParams = {
  'key': _apiKey,
  // skip metacritic filter when browsing by date range
  // — upcoming games have no Metacritic score yet
  if (dates.isEmpty) 'metacritic': '1,100',
  if (dates.isNotEmpty) 'dates': dates,
  // other params...
};

One conditional entry change. ESRB and client-side slug filters remain
active.
```

What I did with this

> *One line change in rawg_service.dart. Tested all three date presets — all returned games. The conditional map entry pattern in Dart became something I used in other parts of the service too.*

1.5 Avatar colour not persisting between sessions

My prompt

```
Saving the avatar colour as a hex string like #00E5FF to Firestore.
When I read it back and try to create a Flutter Color object from it,
int.parse is throwing a FormatException. The string is coming back
from Firestore correctly — I can see it in the console. The
conversion is what is failing.
```

AI response

```
The # prefix is the problem. Dart's int.parse with radix: 16 cannot
handle the
# character — it expects a pure hex string.

Strip the prefix before parsing, then prepend 0xFF for full opacity:

final hex = (data['avatarColor'] as String? ?? '00E5FF')
    .replaceFirst('#', '');
final color = Color(int.parse(hex, radix: 16) + 0xFF000000);

Two options for where to strip it:
Option A — strip on read (above)
Option B — strip on save: store without # to begin with
Either approach works — just be consistent.
```

What I did with this

Used the strip-on-read approach with a null safety fallback defaulting to PlayPal cyan if the field is missing. Avatar colour now persists correctly across sessions and logins.

1.6 Password reset success message not appearing

My prompt

```
The password reset flow calls sendPasswordResetEmail() correctly — I
can verify the email arrives. But the success message I set via
setState never appears on screen. The loading spinner disappears but
the green confirmation text does not show. No error is thrown either.
```

AI response

```
The loading state is being cleared before the success message is set.
Likely
caused by isLoading being set outside the try/catch rather than in a
finally
block. The fix:

try {
  await FirebaseAuth.instance.sendPasswordResetEmail(email: email);
  if (mounted) setState(() => _successMessage = 'Reset email sent');
} on FirebaseAuthException catch (e) {
  if (mounted) setState(() => _errorMessage = e.message);
} finally {
  // finally runs after BOTH try and catch
  // guarantees loading is cleared regardless of outcome
  if (mounted) setState(() => _isLoading = false);
}
```

What I did with this

Added the finally block. Also added mounted checks that were missing in a few other async functions in the same screen. Success message now shows correctly.

1.7 Forum post likes not updating in real time for other users

My prompt

```
When I like a post the count updates immediately on my screen. But if
I have the same forum open in a second browser tab logged in as a
different user, their likeCount does not update — they have to
refresh. The forum uses a StreamBuilder so I expected it to update
automatically.
```

AI response

```
The StreamBuilder is working, but you are likely doing a full array read-
modify-write
for likes instead of using Firestore's atomic operations:

// WRONG — reads the array, adds to it, writes back (race condition):
final current = doc['likes'] as List;
await postRef.update({ 'likes': [...current, uid] });

// CORRECT — arrayUnion is atomic, no race condition:
await postRef.update({
  'likes': FieldValue.arrayUnion([uid]),
  'likeCount': FieldValue.increment(1),
});

arrayUnion adds the uid atomically. The Firestore stream on the other
client
detects the document update and triggers a StreamBuilder rebuild
automatically.
```

What I did with this

Was using the read-modify-write approach. Switched to arrayUnion and arrayRemove for both like and unlike. Tested with two browser tabs — like counts now update in real time across both sessions without any refresh.

1.8 State not shared between two screens in the same tab bar

My prompt

```
I have a tab bar with Discover and Feed tabs. When I save a review on
the Discover tab I want the Feed tab to reflect the new activity
immediately when I switch to it. But the feed only updates when I
restart the app — switching tabs does not trigger a reload.
```

AI response

```
Tabs in a Flutter TabBar only call initState() once on creation — they
don't
rebuild when you switch back to them. Two solutions:

Option 1 — use a StreamBuilder instead of a FutureBuilder for the feed.
If the feed listens to a Firestore stream, it updates automatically when
new
activity is written, regardless of which tab is active.

Option 2 — pass a callback from the parent and call it after saving a
review:
onReviewSaved: () => feedKey.currentState?.reload(),

Option 1 is cleaner for real-time data. Option 2 is simpler if you want
explicit control over when the feed refreshes.
```

What I did with this

Used Option 1 for the activity feed — switched from a one-time Firestore get() to a Stream with StreamBuilder. The feed now reflects new reviews immediately when they are written, whether the tab is active or not.

---

2. Styling and UI Design

PlayPal's visual design was intentional from the start — dark gaming aesthetic with a specific cyan accent colour. I had a clear direction for what I wanted but used Claude to get unstuck on specific Flutter implementation details and to get a second opinion on specific design decisions.

2.1 Choosing the app colour palette

My prompt

```
I want the app to look like a professional dark-themed gaming product
— something in the range of GitHub dark mode but with a strong gaming
accent colour. I need hex values for background layers, surfaces,
borders, primary text, secondary text and an accent. It needs to feel
like a real product not a student project.
```

AI response

```
Based on GitHub Dark as a proven foundation for readability:

Background:       #0D1117  (near black)
Surface (cards):  #161B22  (elevated layer)
Surface 2:        #1C2333  (nested cards, inputs)
Border:           #30363D
Primary text:     #E6EDF3
Secondary text:   #8B949E  (labels, metadata, captions)

Accent — cyan works well for a gaming context, high contrast on dark:
#00E5FF  (main accent — buttons, links, highlights)
#008FAA  (dimmed accent — hover states, secondary)

Supporting semantic colours:
#4ADE80  green — success, Steam features, achievements
#FFD700  gold — trophy badge, achievements, premium feel
#F87171  red — errors, technical challenges, warnings
#A78BFA  purple — calls-to-action, friend requests
#FB923C  orange — notification badges
```

What I did with this

Used this exact palette throughout the entire application, presentation slides, README, and landing page. Defined them as Dart constants at the top of each screen file and referenced them consistently. The surface/surface2 distinction was particularly useful for creating visual hierarchy in nested card layouts.

2.2 Responsive navigation — desktop panel vs mobile drawer

My prompt

```
On desktop I want a fixed 220px left navigation panel always visible.
On mobile it should collapse into a slide-out drawer. I don't want to
duplicate the navigation widget code for both layouts — what is the
cleanest pattern in Flutter?
```

AI response

```
Define the nav widget once, conditionally render it in two places:

final isWide = MediaQuery.of(context).size.width >= 900;

Scaffold(
  key: _scaffoldKey,
  // Drawer only exists on mobile — ignored on desktop
  drawer: isWide ? null : NavPanel(onNavigate: _navigate),
  body: Row(
    children: [
      // Fixed panel on desktop only
      if (isWide) SizedBox(width: 220, child: NavPanel(onNavigate:
_navigate)),
      Expanded(child: _currentScreen),
    ],
  ),
)

Hamburger button triggers: _scaffoldKey.currentState?.openDrawer()
Only show it when !isWide:
if (!isWide) IconButton(icon: Icon(Icons.menu), onPressed: openDrawer)
```

What I did with this

Implemented exactly this pattern. NavPanel is defined once and rendered in both contexts. The 900px threshold was chosen to cover most tablets in portrait and all phones. The scaffoldKey pattern for programmatically opening the drawer from the top bar was the part I needed to look up.

2.3 Game card hover effect and review badge overlay

My prompt

```
I want game cards in the grid to have a subtle border highlight on
hover, and show a coloured status badge in the corner when the user
has reviewed that game. How do overlays and hover states work in
Flutter web?
```

AI response

```
Use Stack for overlays and MouseRegion for hover on web:

MouseRegion(
  onEnter: (_) => setState(() => _hovered = true),
  onExit:  (_) => setState(() => _hovered = false),
  child: Stack(
    children: [
      AnimatedContainer(
        duration: const Duration(milliseconds: 150),
        decoration: BoxDecoration(
          border: Border.all(
            color: _hovered ? const Color(0xFF00E5FF) : const
Color(0xFF30363D),
            width: _hovered ? 1.5 : 0.5,
          ),
          borderRadius: BorderRadius.circular(8),
        ),
        child: GameCardContent(),
      ),
      // Review badge — top right corner
      if (hasReview) Positioned(
        top: 8, right: 8,
        child: StatusBadge(status: reviewStatus),
      ),
    ],
  ),
)
```

What I did with this

Used MouseRegion with AnimatedContainer for the hover effect — the border animates from grey to cyan on hover over 150ms. The status badge uses a Positioned widget in a Stack with a coloured pill showing Playing/Completed/Dropped. Long pressing the card opens the review dialog which pre-fills if a review already exists.

2.4 Building a tappable star rating widget

My prompt

```
I need a star rating input where users tap a star to set their rating
from 1 to 5. No built-in Flutter widget does this. What is the
cleanest way to build it so it looks good and stores as a double for
averaging?
```

AI response

```
Build it with a Row of GestureDetector-wrapped Icons:

double _rating = 0.0;

Row(
  mainAxisSize: MainAxisSize.min,
  children: List.generate(5, (index) {
    final star = index + 1;
    return GestureDetector(
      onTap: () => setState(() => _rating = star.toDouble()),
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 2),
        child: Icon(
          star <= _rating ? Icons.star : Icons.star_border,
          color: star <= _rating ? Colors.amber : Colors.grey,
          size: 28,
        ),
      ),
    );
  }),
)

Store as double (1.0 to 5.0) so averaging works: sum / count gives 4.2
etc.
```

What I did with this

Built exactly this. Stored as double in Firestore which allows displaying average ratings to one decimal place on the profile screen. Tapping a star fills all stars up to and including that position.

2.5 Achievement progress bar turning gold at 100% completion

My prompt

```
The Steam achievements panel has a LinearProgressIndicator showing
completion percentage. I want it to be cyan normally but turn gold
when the player hits 100%. The valueColor property expects an
Animation not a plain Color — not sure how to make it conditional.
```

AI response

```
Use AlwaysStoppedAnimation to wrap a static color in the Animation
interface:

LinearProgressIndicator(
  value: percentage / 100,  // 0.0 to 1.0
  backgroundColor: Colors.grey.withOpacity(0.25),
  valueColor: AlwaysStoppedAnimation<Color>(
    percentage >= 100
      ? const Color(0xFFFFD700)  // gold at 100%
      : const Color(0xFF00E5FF), // cyan otherwise
  ),
  minHeight: 6,
  borderRadius: BorderRadius.circular(3),
)

AlwaysStoppedAnimation wraps a static value in the Animation<Color>
interface
that LinearProgressIndicator requires. It doesn't animate — just
satisfies the type.
```

What I did with this

Used directly. The progress bar turns gold the moment percentage hits 100. Added a small completion label alongside it showing '47 / 50 achievements' so the number is clear without needing to read the bar percentage.

2.6 Consistent card component to avoid duplicating decoration code

My prompt

```
I have card-style containers appearing on many screens — game cards,
review entries, forum posts, friend profile sections. Each was styled
slightly differently because I was writing the Container decoration
inline everywhere. How do I make a reusable card without prop-
drilling becoming a mess?
```

AI response

```
Create a simple AppCard widget with your standard decoration as defaults:

class AppCard extends StatelessWidget {
  final Widget child;
  final Color? borderColor;  // override for highlighted cards
  final EdgeInsets? padding;

  const AppCard({ required this.child, this.borderColor, this.padding });

  @override
  Widget build(BuildContext context) => Container(
    padding: padding ?? const EdgeInsets.all(14),
    decoration: BoxDecoration(
      color: const Color(0xFF161B22),
      border: Border.all(
        color: borderColor ?? const Color(0xFF30363D),
        width: borderColor != null ? 1.0 : 0.5,
      ),
      borderRadius: BorderRadius.circular(8),
    ),
    child: child,
  );
}

Usage: AppCard(child: ReviewContent(), borderColor: statusColour)
```

What I did with this

Created AppCard and went back through existing screens replacing duplicated Container decorations. Cards are now visually consistent. The borderColor override is used for review cards where the border matches the play status colour — green for Completed, orange for Playing, red for Dropped.

2.7 Mobile navigation bar overflowing at 375px

My prompt

```
Even after reducing the nav bar to three buttons, iPhone SE at 375px
is still overflowing. The buttons seem to need more horizontal space
than is available even with just three. I need a secondary scaling
breakpoint for very narrow screens.
```

AI response

```
Add a second breakpoint at 400px and scale button sizes and padding:

final screenWidth = MediaQuery.of(context).size.width;
final isNarrow = screenWidth < 400; // iPhone SE (375px) threshold

IconButton(
  icon: Icon(icon, size: isNarrow ? 18.0 : 20.0),
  padding: EdgeInsets.symmetric(
    horizontal: isNarrow ? 6.0 : 10.0,
    vertical:   isNarrow ? 6.0 : 8.0,
  ),
  // CRITICAL: override Flutter's default 48x48 minimum touch target
  // Without this the buttons still overflow even with padding reduced
  constraints: const BoxConstraints(),
  onPressed: onTap,
)

The constraints override is the key — IconButton enforces a minimum
48x48px
touch target by default which causes overflow on 375px screens.
```

What I did with this

The constraints override was the thing I was missing — had no idea Flutter imposed a minimum touch target on IconButton. That single line fixed the overflow that padding reduction alone could not solve. Tested in Chrome DevTools at 375px and on an actual iPhone SE.

2.8 Login screen split layout — decorative panel and form

My prompt

```
I want the login screen to have a split layout on desktop —
decorative panel on the left showing feature highlights, form on the
right. On mobile the left panel should disappear and the form takes
full width. What is the best Flutter structure for this?
```

AI response

```
Use a Row with conditional rendering based on screen width:

final isWide = MediaQuery.of(context).size.width >= 800;

Row(
  children: [
    // Left decorative panel — hidden on narrow screens
    if (isWide) Expanded(
      flex: 1,
      child: DecorativePanel(), // background gradient, feature list etc
    ),
    // Right form — always shown
    Expanded(
      flex: isWide ? 1 : 1,  // takes full width when panel is hidden
      child: LoginForm(),
    ),
  ],
)

800px threshold works well — hides the panel on tablets in portrait and
phones
while keeping it on most laptop and desktop screens.
```

What I did with this

Implemented the split layout with 800px threshold. The left panel shows the PlayPal logo, tagline and a short feature list. The right side has the email/password form, forgot password link and sign-in button. Below 800px the panel disappears and the form centres on the screen.

---

3. Implementation Help

These entries cover situations where I had a clear idea of what I wanted to build but hit a specific technical wall — either not knowing the correct API to use, or encountering an edge case in Firebase or Flutter that was not obvious from documentation alone.

3.1 Steam API CORS — proxying through Cloud Functions

My prompt

```
I want to import users' Steam library into PlayPal. Direct calls to
the Steam API from Flutter web are blocked immediately with a CORS
error before the request even reaches Steam. I already use Firebase —
is there a Firebase-native way to proxy these calls server-side?
```

AI response

```
Steam doesn't set Access-Control-Allow-Origin. Browsers block all direct
calls.
This can't be fixed client-side. You need server-side execution.

Firebase Cloud Functions are the right solution — they run Node.js
server-side
where browser CORS policies don't apply. Store the API key as a Firebase
Secret:

// functions/index.js
const STEAM_KEY = defineSecret('STEAM_API_KEY');

exports.getSteamGames = functions.https.onCall(
  { secrets: [STEAM_KEY], cors: true },
  async (data) => {
    const key = process.env.STEAM_API_KEY; // injected at runtime, never
in code
    const result = await fetchUrl(
      `${STEAM_BASE}/IPlayerService/GetOwnedGames/v1/?key=$
{key}&steamid=${id}`
    );
    return { games: result.response.games };
  }
);

Flutter calls the function over HTTPS. Function calls Steam. No CORS
issue.
```

What I did with this

Built three Cloud Functions — getSteamProfile, getSteamGames, getSteamAchievements. The achievements function required calling two Steam endpoints in parallel and merging results using Promise.all. The Steam ID parsing to handle three URL formats (numeric, full URL, vanity) was my own logic. Section 8.5 and Appendix E.

3.2 Adult content filtering — reliable multi-layer approach

My prompt

```
The game grid is showing adult games which is not acceptable for a
project showcase demonstration. RAWG's ESRB filter alone is not
catching everything. What combination of filters actually works
reliably?
```

AI response

```
Layer multiple filters — no single filter is 100% reliable on RAWG:

Layer 1 — ESRB at the API level (catches explicitly rated content):
'esrb_rating': 'everyone,everyone-10-plus,teen,mature'

Layer 2 — Metacritic score requirement (most effective in practice):
'metacritic': '1,100'
Adult games almost never have Metacritic scores. This single filter
removes
the vast majority of problematic content that gets through the ESRB
filter.

Layer 3 — Client-side slug check (final safety net):
results.where((g) {
  final slug = g['esrb_rating']?['slug'] ?? '';
  return slug != 'adults-only';
}).toList()

Important: make Layer 2 conditional — skip it when dates param is active.
Upcoming games have no Metacritic scores yet and would be incorrectly
filtered.
```

What I did with this

Implemented all three layers. The Metacritic filter was far more effective than expected. The conditional approach for Layer 2 was discovered later when Coming Soon stopped working. Section 8.3.

3.3 Recently viewed — preventing duplicates using document ID as key

My prompt

```
I want a recently viewed strip on the home screen. If the user opens
the same game twice it should appear once and move to the front, not
create a duplicate entry. Limiting to the last 10 without doing
constant deletes would also be good.
```

AI response

```
Use the gameId as the Firestore document ID. Writing to the same doc ID
overwrites it — automatic deduplication. Timestamp updates so it floats
to top.

await _db.collection('users').doc(uid)
    .collection('recentlyViewed')
    .doc(game.id.toString())  // gameId as doc ID = no duplicates
    .set({
  'gameId': game.id,
  'name': game.name,
  'backgroundImage': game.backgroundImage,
  'timestamp': FieldValue.serverTimestamp(),
}, SetOptions(merge: false)); // overwrite completely

Cap at 10 by always querying with .limit(10) ordered by timestamp.
Older entries remain in Firestore but are never fetched or displayed.
No deletion logic needed at all.
```

What I did with this

Implemented in user_service.dart. The gameId-as-docId pattern was the insight — had been thinking about this as an array management problem when it's simpler as a map. The recently viewed strip now correctly moves a game to the front when revisited.

3.4 Friend requests — writing across the user security boundary

My prompt

```
Sending a friend request needs to write to the recipient's
friendRequests subcollection. My Firestore security rules block
cross-user writes by default. How do I allow this one specific case
without making all user subcollections publicly writable?
```

AI response

```
Add an explicit rule for friendRequests that overrides the default
restriction:

// Default — owner only:
match /users/{userId} {
  allow write: if request.auth.uid == userId;
}

// Exception — any authenticated user can send a friend request:
match /users/{userId}/friendRequests/{requestId} {
  allow read:  if request.auth.uid == userId;  // only recipient reads
  allow write: if request.auth != null;          // any authed user
writes
}

This is narrowly scoped to friendRequests only. All other subcollections
(reviews, favourites, friends) remain owner-write only.
The most specific matching rule wins in Firestore.
```

What I did with this

Added the explicit friendRequests rule. Tested by trying to write to a different user's reviews subcollection — correctly got permission-denied. Friend requests work. Understanding that Firestore uses most-specific-rule-wins was the key concept that made this clear.

3.5 Loading three Firestore collections in parallel on profile open

My prompt

```
The profile screen needs to load the user document, the reviews
subcollection, and the favourites subcollection on open. Currently
running them sequentially which makes the screen feel slow. Is there
a Dart equivalent of Promise.all?
```

AI response

```
Future.wait fires multiple futures simultaneously:

Future<void> _loadProfile() async {
  setState(() => _isLoading = true);

  final results = await Future.wait([
    _db.collection('users').doc(uid).get(),
    _db.collection('users').doc(uid).collection('reviews').get(),
    _db.collection('users').doc(uid).collection('favourites').get(),
  ]);

  final profileDoc  = results[0] as DocumentSnapshot;
  final reviewsSnap = results[1] as QuerySnapshot;
  final favsSnap    = results[2] as QuerySnapshot;

  // calculate stats from reviews client-side:
  final completed = reviewsSnap.docs.where((d) =>
d['status']=='Completed').length;
  final avgRating = reviewsSnap.docs.isEmpty ? 0.0
      : reviewsSnap.docs.map((d) => d['personalRating'] as double)
          .reduce((a,b) => a+b) / reviewsSnap.docs.length;
}

Sequential: 3 reads × ~200ms = ~600ms. Parallel: ~200ms total.
```

What I did with this

Replaced sequential reads with Future.wait on both the profile screen and the friend profile screen. Noticeable improvement in how quickly stats appear after the screen opens.

3.6 Username change — multi-step atomic update

My prompt

```
Changing a username requires updating the user document, creating a
new usernames lookup entry, and deleting the old one. If it fails
halfway through, the app could end up in an inconsistent state where
the old username is gone but the new one is not reserved yet. How do
I handle this safely without a full transaction?
```

AI response

```
Sequence the operations to fail safely if anything goes wrong:

Future<void> changeUsername(String newName) async {
  final lower = newName.toLowerCase();
  final oldLower = _profile['usernameLower'] as String;

  // Step 1: check availability first before touching anything
  final exists = await _db.collection('usernames').doc(lower).get();
  if (exists.exists) throw Exception('Username already taken');

  // Step 2: delete old reservation
  await _db.collection('usernames').doc(oldLower).delete();

  // Step 3: create new reservation
  await _db.collection('usernames').doc(lower).set({'uid': uid});

  // Step 4: update user document
  await _db.collection('users').doc(uid)
      .update({ 'username': newName, 'usernameLower': lower });
}

If step 3 or 4 fails the old username is already deleted. Add error
handling
to attempt re-creating the old reservation on failure if needed.
```

What I did with this

Implemented the four-step flow with error handling. In practice step 1 catches almost all conflicts before anything is written. Added a user-visible error message when the name is taken. Username changes work reliably without leaving the app in an inconsistent state.

3.7 Infinite scroll — loading the next page at 80% scroll depth

My prompt

```
I want the game grid to automatically fetch the next RAWG page when
the user reaches 80% of the scroll depth. I don't want a Load More
button — it should happen automatically without the user doing
anything. How does the ScrollController approach work?
```

AI response

```
Attach a ScrollController and listen for scroll position changes:

late final ScrollController _scrollController;

@override
void initState() {
  super.initState();
  _scrollController = ScrollController();
  _scrollController.addListener(_onScroll);
  fetchGames(); // initial load
}

void _onScroll() {
  final pos = _scrollController.position;
  final threshold = pos.maxScrollExtent * 0.80;
  if (pos.pixels >= threshold && !_isLoadingNextPage) {
    fetchGames(nextPage: true);
  }
}

@override
void dispose() {
  _scrollController.dispose(); // always dispose
  super.dispose();
}

Pass the controller to your GridView: controller: _scrollController
```

What I did with this

Implemented exactly this. The _isLoadingNextPage guard prevents firing multiple simultaneous requests if the user scrolls quickly. Used dispose() to clean up the controller. Infinite scroll works smoothly — the next page appends to the existing list with no visible jump.

3.8 Steam achievements — calling two endpoints and merging the results

My prompt

> The achievement panel needs to show each achievement with its display name, icon, and whether the user has unlocked it. I can see Steam has two separate endpoints for this — GetPlayerAchievements gives unlock data but no names, GetSchemaForGame gives names and icons but no unlock data. How do I combine them efficiently?

AI response

```
Call both endpoints simultaneously with Promise.all, then merge using a
lookup map:

const [playerResult, schemaResult] = await Promise.all([
  fetchUrl(`${STEAM_BASE}/ISteamUserStats/GetPlayerAchievements/v1/
           ?key=${key}&steamid=${steamId}&appid=${appId}`),
  fetchUrl(`${STEAM_BASE}/ISteamUserStats/GetSchemaForGame/v2/
           ?key=${key}&appid=${appId}&l=english`),
]);

// Build a lookup map from schema data keyed by achievement API name
const schemaMap = {};
schemaAchievements.forEach(a => {
  schemaMap[a.name] = {
    displayName: a.displayName ?? a.name,
    icon:        a.icon ?? '',
    iconGray:    a.icongray ?? '',
  };
});

// Merge — player data contributes achieved/timestamp,
// schema map contributes name and icon
const merged = playerAchievements.map(a => ({
  displayName: schemaMap[a.apiname]?.displayName ?? a.apiname,
  achieved:    a.achieved === 1,
  unlockedAt:  a.unlocktime ? new Date(a.unlocktime * 1000).toISOString()
: null,
  icon:        schemaMap[a.apiname]?.icon ?? '',
  iconGray:    schemaMap[a.apiname]?.iconGray ?? '',
}));

Why Promise.all: calling sequentially would take ~600ms. In parallel
~300ms.
Why a map not a loop: O(1) lookup per achievement vs O(n) scan = much
faster.
```

What I did with this

> *Built getSteamAchievements as the third Cloud Function. The schemaMap pattern was the insight I needed — had been thinking about this as two loops rather than a build-once lookup. The sorting logic (unlocked first, then locked) and the percentage calculation were my own additions. The progress bar turning gold at 100% used AlwaysStoppedAnimation on the Flutter side which I found separately.*

3.9 Steam — getSteamGames data processing on the server side

My prompt

```
Once getSteamGames gets the library back from Steam it is raw data —
playtime in minutes, icon hashes instead of URLs, no sorting. I need
to send Flutter a clean sorted list with hours already calculated and
full image URLs constructed. Is it better to do this processing in
the Cloud Function or in Flutter?
```

AI response

```
Do it in the Cloud Function. Send Flutter clean data — never raw API
responses.
Flutter should not need to know Steam's internal data format.

// Sort by playtime descending — most played game first
const sorted = games.sort((a, b) => b.playtime_forever -
a.playtime_forever);

// Calculate totals
const totalMinutes = games.reduce((sum, g) => sum + g.playtime_forever,
0);
const totalHours   = Math.round((totalMinutes / 60) * 10) / 10;
const playedGames  = games.filter(g => g.playtime_forever > 0).length;

// Transform each game — construct full image URLs from Steam CDN pattern
return {
  totalGames, totalHours, playedGames,
  games: sorted.map(g => ({
    appId:         g.appid,
    name:          g.name,
    playtimeHours: Math.round((g.playtime_forever / 60) * 10) / 10,
    // Steam icon URL is constructed from appId + icon hash
    iconUrl:    g.img_icon_url
      ?
`https://media.steampowered.com/steamcommunity/public/images/apps/
        ${g.appid}/${g.img_icon_url}.jpg` : '',
    // Header image URL follows a predictable pattern — just needs appId
    headerUrl:  `https://cdn.cloudflare.steamstatic.com/steam/apps/
                ${g.appid}/header.jpg`,
  }))
};
```

What I did with this

*Implemented all the processing server side. The Math.round((x / 60) * 10) / 10 pattern for one decimal place rounding was something I had to work through step by step — divide by 60, multiply by 10, round, divide by 10. The header image URL pattern I found by inspecting Steam URLs manually. The icon URL pattern needed the appId and the hash combined which was not obvious from the API docs.*

---

4. Learning Sessions

Flutter and Firebase were both completely new at the start of this project. These entries reflect structured study sessions where I wanted to verify I understood specific patterns correctly before building on top of them. The format was: explanation, then quiz questions to confirm understanding before moving on.

4.1 StatefulWidget lifecycle — initState, setState, dispose, mounted

My prompt

```
Can you walk me through the StatefulWidget lifecycle? I know the
words but I am not confident on the order things happen in or why
mounted needs to be checked before setState. I keep copying the
pattern without being sure I understand it.
```

AI response

```
Lifecycle order — always in this sequence:

1. initState() — runs once when the widget first enters the tree.
   Call your data loading functions here, never in build().
   Always call super.initState() first.

2. build() — called after initState, then after every setState().
   Returns the widget tree. Should be pure — no side effects.

3. setState() — tells Flutter to call build() again with new data.
   Only call inside the State class. Triggers a rebuild.

4. dispose() — called when the widget leaves the tree (user navigates
away).
   Clean up TextEditingControllers, animation controllers, etc here.
   Always call super.dispose() last.

Why mounted matters:
If an async operation is in flight and the user navigates away, dispose()
runs.
When the async operation completes, calling setState() on a disposed
widget crashes.
mounted is false after dispose() — checking it prevents the crash:
if (mounted) setState(() { data = result; });
```

What I did with this

Added mounted checks to every async function across all screens. Found several places where they were missing. This also explained why I was occasionally seeing 'setState called during build' errors — the pattern became clear and I fixed those as well.

4.2 StreamBuilder vs FutureBuilder — when to use each

My prompt

```
I have been using FutureBuilder for everything and some features are
not updating in real time as I expected. I understand Future means
one value but I am not clear on when StreamBuilder is the right
choice versus just reloading on a timer or pulling to refresh.
```

AI response

```
FutureBuilder — use when data arrives once and does not change
spontaneously.
Examples: RAWG API call, loading a user profile on screen open.
The builder runs once when the future completes, then stops.

StreamBuilder — use when data can change over time and the UI should
update
automatically. The builder re-runs every time the stream emits a new
value.
Examples: Firestore .snapshots() on a forum collection.

Firestore-specific: .get() returns a Future. .snapshots() returns a
Stream.
If you use FutureBuilder with .get() on a forum, posts only load once.
If you use StreamBuilder with .snapshots(), posts appear in real time.

Two places in PlayPal that require StreamBuilder:
1. AuthGate — listens to authStateChanges() for automatic login/logout
routing.
2. Forum screen — listens to forum posts so they appear without refresh.

Rule of thumb: if the UI should react to data changing — use a Stream.
```

What I did with this

Changed the forum screen from FutureBuilder with .get() to StreamBuilder with .snapshots(). Posts now update in real time. Changed the AuthGate to StreamBuilder on authStateChanges() — eliminated the need for any manual Navigator.push calls after login.

4.3 Firestore security rules — do they cascade to subcollections?

My prompt

```
I want authenticated users to be able to read any profile but only
write to their own data. I assumed the rule on the parent document
would apply to its subcollections but that doesn't seem to be how it
works. Do rules cascade?
```

AI response

```
No — rules do NOT cascade from parent to subcollection in Firestore.
Each match block applies only to the exact path it matches.

match /users/{userId} {
  allow read:  if request.auth != null;       // any authed user reads
  allow write: if request.auth.uid == userId; // owner only writes
  // This does NOT apply to subcollections
}

// Subcollections need their own rules:
match /users/{userId}/reviews/{reviewId} {
  allow read:  if request.auth != null;
  allow write: if request.auth.uid == userId;
}

// Exception for friendRequests — cross-user writes needed here:
match /users/{userId}/friendRequests/{requestId} {
  allow read:  if request.auth.uid == userId;
  allow write: if request.auth != null;
}

The most specific matching rule wins when there is overlap.
```

What I did with this

Rewrote the security rules with explicit rules per subcollection. Tested all cases manually — reading another user's profile works, writing to their reviews fails, friend requests work. Full rules in Appendix D.

4.4 FieldValue.serverTimestamp vs DateTime.now() for timestamps

My prompt

```
I have been using DateTime.now() for timestamps on reviews and forum
posts. I noticed the report mentions serverTimestamp but I am not
sure what the actual practical difference is — both seem to order
things correctly in my tests.
```

AI response

```
DateTime.now() uses the device clock which can be:
- Set incorrectly by the user
- In a different timezone
- Slightly out of sync across devices

FieldValue.serverTimestamp() writes the Firebase server time when the
document
is saved — consistent regardless of which client device writes the
document.

For the activity feed this matters significantly. The feed orders
activity by
timestamp DESC. If two users submit reviews at the same second but one
device
has a clock that's 5 minutes ahead, the ordering in the feed could appear
wrong.

Rule of thumb:
Use serverTimestamp() for any field you will ORDER BY in queries.
DateTime.now() is acceptable for local display ('posted just now') only.

Note: Firestore writes the actual value slightly after the client
receives the
document — the field shows null locally until the server confirms. Handle
this by checking for null before displaying timestamps.
```

What I did with this

Switched all review, forum post, and activity document timestamps to FieldValue.serverTimestamp(). Added null checks when displaying timestamps in case the server hasn't confirmed yet. Feed ordering is now consistent across different device clocks.

4.5 Understanding Uri.https vs string concatenation for API URLs

My prompt

```
I noticed the RAWG service uses Uri.https to build URLs instead of
just concatenating strings. I changed it to string concatenation in a
test and searches with apostrophes stopped working. Can you explain
exactly what Uri.https does differently and why it matters?
```

AI response

```
String concatenation builds a raw URL string — special characters break
it:

// WRONG — apostrophe in 'Demon's Souls' breaks the URL
final url = 'https://api.rawg.io/api/games?search=Demon's Souls';
// URL becomes malformed — browser/HTTP client may reject it

Uri.https() builds the URL and percent-encodes special characters
automatically:

final uri = Uri.https('api.rawg.io', '/api/games', {
  'search': "Demon's Souls",  // apostrophe handled automatically
});
// Produces: https://api.rawg.io/api/games?search=Demon%27s+Souls

Characters that get encoded: spaces → %20 or +, ' → %27, & → %26,
= → %3D, # → %23, and many others.

Also safer for query parameter injection — prevents a user from breaking
the URL by typing special characters into the search field.
```

What I did with this

This explained why some searches were failing silently — the apostrophe in game names was corrupting the URL before it reached RAWG. Kept Uri.https throughout the service. Also understood why the conditional map entries are cleaner than conditional string concatenation.

4.6 Understanding Promise.all and why it halves Steam achievement load time

My prompt

```
In index.js we use Promise.all to call both Steam endpoints
simultaneously. I get that it's faster but I want to understand
mechanically what is actually happening — are the two requests
running on different threads? And when both resolve, how does
Promise.all know to continue?
```

AI response

```
Node.js is single-threaded but uses an event loop for async I/O. Both
requests
are NOT running on different threads — they're running on the same thread
but
both are waiting on network I/O simultaneously.

Think of it like this:
You send letter 1 to Steam (GetPlayerAchievements) and immediately
send letter 2 (GetSchemaForGame) before letter 1 has replied.
Both are now 'in the post' at the same time.
Steam receives both, processes both, sends back both replies.
You wait for whichever takes longer — not for each one sequentially.

Promise.all takes an array of Promises and returns a new Promise that:
- Resolves when ALL input Promises resolve (with array of results)
- Rejects immediately if ANY input Promise rejects

const [r1, r2] = await Promise.all([fetchUrl(url1), fetchUrl(url2)]);
// r1 = result of url1, r2 = result of url2
// Code below this line runs only when BOTH have completed

Sequential: 300ms + 300ms = 600ms. Parallel: max(300ms, 300ms) = 300ms.
```

What I did with this

This clarified the mental model — it is not about threads but about not waiting idle. I could explain this correctly in the presentation technical Q&A. The Promise.all / Future.wait parallel pattern became something I looked for and applied wherever multiple independent async calls were being made.

4.7 The schemaMap pattern — why not just loop through the array each time?

My prompt

```
In getSteamAchievements we build schemaMap before merging. I
understand it's faster but I want to understand the actual difference
in terms of the number of operations — when does the approach without
the map start becoming a real problem?
```

AI response

```
Without schemaMap — for every player achievement, scan the full schema
array:

playerAchievements.forEach(player => {
  // scan entire schema array until we find the matching name
  const schema = schemaAchievements.find(s => s.name === player.apiname);
});

If a game has 100 achievements:
- Best case: first item in schema matches every time = 100 comparisons
- Worst case: last item matches every time = 100 × 100 = 10,000
comparisons
- Average: ~50 × 100 = 5,000 comparisons
This is O(n²) — performance degrades quadratically with achievement
count.

With schemaMap — build map once, then each lookup is O(1):

// Build: 100 operations (one forEach over schema)
schemaAchievements.forEach(a => { schemaMap[a.name] = {...}; });

// Merge: 100 operations (one for each player achievement)
playerAchievements.map(a => ({ ...schemaMap[a.apiname] }));

Total: 200 operations. Always. Regardless of achievement count.
This is O(n) — linear. For 1000 achievements: 200 vs ~500,000
comparisons.
```

What I did with this

The $O(n^2)$ vs $O(n)$ explanation made the performance difference concrete rather than abstract. I could explain the schemaMap pattern in the presentation specifically using the operation count comparison. This also became a pattern I looked for in other merging scenarios.

---

5. Presentation and Documentation

In the weeks before the final presentation I used Claude to help with the demo video script, build the slide deck iteratively, and assist with final report proofreading and citation checks.

5.4 Report proofreading — spelling, punctuation, grammar

My prompt

```
Can you proofread this section of my report for spelling mistakes,
punctuation errors and grammar issues only. Do not rewrite anything
or change the meaning — just flag the actual errors: [pasted section
of report text]
```

AI response

```
Returned the same text with corrections flagged — examples from multiple
passes:

'platoform' → 'platform'
'Steam Web api' → 'Steam Web API'  (API should always be uppercase)
'technical decisin' → 'technical decision'
'it's' → 'its'  (possessive, not contraction)
'alot' → 'a lot'  (two words)
Missing commas after introductory clauses in three sentences.
Missing comma before 'which' in non-restrictive clause.
Each correction was noted with a brief explanation of why it was wrong.
```

What I did with this

Went through each correction individually and accepted only clear errors. Rejected any suggestions that altered sentence structure or phrasing — those were not asked for. Ran six sections through this process before final submission. The 'its vs it's' and 'API capitalisation' corrections were the most frequent.

5.5 MoSCoW citation in Harvard format

My prompt

```
Who created the MoSCoW prioritisation technique and when was it
published? I need a proper Harvard reference for the report.
```

AI response

```
MoSCoW was introduced by Dai Clegg in 1994.

Harvard format:
Clegg, D. and Barker, R. (1994). Case Method Fast-Track: A RAD Approach.
Wokingham: Addison-Wesley.
```

What I did with this

Used this citation wherever MoSCoW is referenced in the report. Checked that the publisher and place of publication were correct before including it in the references section.

5.6 RAMP methodology — verifying my description before submitting

My prompt

```
I am writing the methodology section and want to make sure my
description of RAMP is accurate before submitting. Can you confirm
what each letter stands for and give a brief explanation of how each
phase is supposed to work?
```

AI response

```
RAMP: Research, Analysis, Model, Plan.

Research — investigation of the problem domain. Literature review,
competitor
analysis, technology comparison. Informs requirements.

Analysis — translating research findings into concrete requirements.
MoSCoW prioritisation. Functional and non-functional requirements
defined.

Model — architectural and design decisions. System architecture, data
model,
wireframes, security design. Done before coding begins.

Plan — sprint structure, timeline, risk register, mitigation strategies.

Important note: phases are rarely perfectly sequential in practice.
Requirements are frequently revised as technical constraints emerge
during
implementation — this is normal and worth acknowledging in the report.
```

What I did with this

Verified the description in section 3 was accurate. The note about phases not being perfectly sequential was something I included because it was genuinely true of how the project developed — the Firestore data model was revised twice during implementation as query requirements changed.